
AUFGABE ZUR VORLESUNG

Atomare Datei-Operationen – Ihren Storage absturzsicher machen

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema Atomarität, fsync, Sperren, Lost Update, Dual Write
Dozent Dr. Ulrich Winkler
Semester 5. Semester · Informationstechnik

Voraussetzung: Übung 01 (`01_parcel_service`) – Sie haben eine laufende API mit einem File Storage unter `$DATA_DIR`.

Ziel: Den Storage Ihrer bestehenden API so umbauen, dass er einen Absturz mitten im Schreiben übersteht. Sie schreiben in dieser Übung **keine neue Anwendung**

Warum

In Übung 01 haben Sie gebaut, *was* gespeichert wird. Sie haben dabei stillschweigend angenommen, dass ein Schreibvorgang entweder ganz passiert oder gar nicht. Diese Annahme ist leider nicht immer zutreffend.

`cheatsheet-atomar.md` in diesem Ordner zeigt die Rezepte dagegen. Diese Aufgabe bringt sie in Ihren Code.

Audit Ihres eigenen Codes

Gehen Sie Ihren `store.py` (oder wie Sie das Modul genannt haben) Zeile für Zeile durch und suchen Sie gezielt nach diesen fünf Mustern:

Truncating **Write.** Jedes `open(pfad, "w")`, `Path.write_text()`, `json.dump(obj, open(...))`. Alle drei kürzen die geöffnete Datei zuerst.

Read-Modify-Write ohne Sperre. Jede Stelle, an der Sie eine Datei lesen, etwas ändern und zurückschreiben. Das ist der *Lost-Update-Kandidat*, und `write_atomic` hilft dort **nicht** – es macht jeden einzelnen Write ganz, nicht die Folge aus drei Schritten. (denken Sie an konkurrierende Prozesse)

Zwei Dateien, ein Request. Jede Stelle, an der ein Request zwei Dateien anfasst (typisch: `parcels/<id>.json` und `tracking/<id>.log`). Markieren Sie das Absturzfenster dazwischen.

Anlegen ohne Inhalt. Jedes `O_CREAT | O_EXCL`, gefolgt vom Schreiben des Inhalts.

Fehlendes fsync. Jede Stelle, an der Sie antworten, bevor die Daten dauerhaft sind.

Zum Selbststudium: Threads in Python

Diese Übung dreht sich um **Prozesse** – vier Unicorn-Worker, die sich eine Datei teilen. Die Ebene darunter fehlt Ihnen dann noch: **Threads**. Arbeiten Sie sich selbständig ein, es sind zwei bis drei Stunden.

Der Grund, warum das hierher gehört: `fcntl.flock` und `threading.Lock` lösen dasselbe Problem auf zwei verschiedenen Ebenen, und wer den Unterschied nicht kennt, greift zum falschen

Werkzeug. Das ist keine theoretische Gefahr – der Griff daneben fällt beim Entwickeln mit `--reload` (ein Prozess) nicht auf und schlägt erst unter `--workers 4` zu.

Was Sie sich ansehen sollten

Thema	Einstieg
Threads starten und einsammeln	<code>threading.Thread</code> , <code>.start()</code> , <code>.join()</code>
Die Race Condition selbst bauen	zwei Threads, die <code>x += 1</code> millionenfach ausführen
Sperren	<code>threading.Lock</code> als Kontextmanager (<code>with lock:</code>)
Wiedereintritt	<code>threading.RLock</code> – wann brauchen Sie das statt <code>Lock</code> ?
Bequemere Schnittstelle	<code>concurrent.futures.ThreadPoolExecutor</code>
Warum Threads in Python selten schneller sind	das GIL

Dokumentation: <https://docs.python.org/3/library/threading.html> und <https://docs.python.org/3/library/concurrent.futures.html>

Warum Threads aus der Mode gekommen sind – und warum Sie sie trotzdem verstehen müssen

In den Neunzigern und Zweitausendern war ein Thread pro Request die Standardantwort. Heute ist sie es nicht mehr, aus vier Gründen:

Entwicklung	Was an die Stelle der Threads trat
async / await	Für I/O-lastige Arbeit reicht ein Thread mit einer Event-Loop. Kein Kontextwechsel, keine Sperren – und genau darauf setzt Uvicorn auf oder <code>node.js/javascript</code>
Horizontale Skalierung	Statt mehr Threads im Prozess nimmt man mehr Prozesse, Container, Pods. Hier mit <code>--workers 4</code> ist diese Antwort im Kleinen.
Stateless Services	Der gemeinsame Zustand wandert nach außen – in Datenbank, Cache oder Objektspeicher. Was nicht geteilt wird, muss nicht gesperrt werden.
Serverless	Eine AWS-Lambda-Instanz bearbeitet einen Aufruf zur Zeit. Mehr Last heißt mehr Instanzen, nicht mehr Threads. Geteilter Speicher existiert gar nicht.

Bei Python kam die Eigenart dazu, dass das **GIL** Threads für rechenlastige Arbeit ohnehin unattraktiv machte. Für I/O taugen sie weiter, und genau dort wurden sie von `async / await` abgelöst.

Hinweis: Zwei Lambda-Aufrufe, die gleichzeitig dasselbe Objekt in S3 ändern: lesen, ändern, zurückschreiben. Derselbe *Lost Update* wie oben, nur dass Ihnen diesmal kein `fcntl.flock` hilft – es gibt keinen gemeinsamen Kernel mehr, an dessen Inode eine Sperre hängen könnte. Übrig bleibt, was in dieser Übung das Versionsfeld war: eine Bedingung, die der Speicher selbst prüft. Bei S3 heißt das `If-Match` auf den ETag, bei DynamoDB `ConditionExpression`, bei Ihrer API `412 Precondition Failed`.

Serverless nimmt Ihnen also die Threads ab – und zwingt Sie im selben Zug zu **optimistischer Nebenläufigkeit**, weil pessimistisches Sperren dort gar nicht mehr zur Verfügung steht. Das ist der Grund, warum diese Übung nicht historisch ist: Sie lernen hier am kleinen Beispiel, was Ihnen in jeder verteilten Architektur wieder begegnet, nur ohne die Möglichkeit, es mit einer Sperre zuzudecken.

Die Frage, auf die es hinausläuft

Wenn Sie durch sind, beantworten Sie diese drei – sie sind der Anschluss an das, was Sie oben gebaut haben:

1. Warum schützt `threading.Lock` Ihre Zählung bei `--reload`, aber nicht bei `--workers 4`? Was genau teilen sich vier Worker, und was nicht?
2. `fcntl.flock` funktioniert *auch* zwischen Threads eines Prozesses. Warum nimmt man trotzdem `threading.Lock`, wenn es nur um Threads geht?
3. Ihre API läuft mit vier Workern, und jeder Worker bearbeitet Requests in mehreren Threads. Welche der beiden Sperren brauchen Sie – und warum beantwortet sich die Frage nicht mit „beide“, sondern mit einem Blick darauf, was geschützt werden muss?